

CQRLOGIMPROVED Qt6

Complete SRC Build, Installation, Data Migration and Launch Manual

Qt 6.12.0 - CqrlogImproved CI_(144)_ - MariaDB - Hamlib - Qt6Pas

Recorded from a successful 71/71 = 100% build

Date: 2026-08-21

Core rule: The CqrlogImproved final runtime, Qt6/Qt6Pas, MariaDB, Hamlib and the runtime library chain are built from SRC. apt, snap and flatpak are not used. System tools used only to perform the build (for example shell, make/cmake, Python and Perl) are bootstrap tools and are not part of the CqrlogImproved final runtime.

Contents

1. What this manual records
 2. Final directory layout
 3. SRC component and version list
 4. Before building: safe checks
 5. Build environment and paths
 6. Checking the SRC collection with Python
 7. Build order and dependency chain
 8. Building Qt 6.12.0 from SRC
 9. Building Qt6Pas against the private Qt6
 10. OpenSSL and MariaDB chain
 11. Building Hamlib from SRC
 12. Building CqrlogImproved Qt6
 13. Installing into the private runtime
 14. First launch
 15. Migrating old CQRLOG data and ALL configuration
 16. Integrating into the Amateur Radio Programs menu
 17. Final verification
 18. Troubleshooting
 19. Update procedure
 20. SRC provenance audit
- Appendix A. Final 71/71 stages
- Appendix B. Restoring backups

1. What this manual records

This manual records the practical sequence that produced a working CqrlogImproved Qt6 installation in an isolated private runtime, followed by lossless migration of the existing log databases and application configuration.

- Final binary: ~/Programmas/cqrlog-src/runtime/bin/cqrlog
- Final Qt: 6.12.0
- CqrlogImproved: CI_(144)_, commit b4b766c186d95dbaad3da732527c9cfa6f581976
- Lazarus build tool: 4.4; FPC build tool: 3.2.2
- Data directory after migration: ~/.config/cqrlog
- Old environment: ~/.config/cqrlog2
- The new launcher is placed in the Amateur Radio Programs menu.
- The native white Qt6 appearance is retained as the intended CqrlogImproved base design.

Important precision note: This manual does not invent command history. Command blocks that were explicitly recorded during the successful work are reproduced in full. For early dependencies where one final exact command block was not retained, the manual gives the safe build-system pattern, the pinned version, the intended prefix and the expected artifact.

2. Final directory layout

```
export BASE="$HOME/Programmas/cqrlog-src"
export SYSROOT="$BASE/sysroot"
export TOOLCHAIN="$BASE/toolchain"
export RUNTIME="$BASE/runtime"
export BUILD="$BASE/build"
export SRC="$HOME/CqrlogImproved-SRC"
export CQR="$HOME/CqrlogImproved"

mkdir -p "$SYSROOT" "$TOOLCHAIN" "$RUNTIME" "$BUILD" "$SRC"
```

Expected result / what should appear: The main work directories must exist under ~/Programmas/cqrlog-src, with separate SRC and CqrlogImproved source trees.

```
printf 'BASE=%s\nSYSROOT=%s\nTOOLCHAIN=%s\nRUNTIME=%s\nBUILD=%s\nSRC=%s\nCQR=%s\n' \
"$BASE" "$SYSROOT" "$TOOLCHAIN" "$RUNTIME" "$BUILD" "$SRC" "$CQR"
```

Expected result / what should appear: All paths should be absolute. Example:

```
BASE=/home/yl3gbc/Programmas/cqrlog-src
RUNTIME=/home/yl3gbc/Programmas/cqrlog-src/runtime
CQR=/home/yl3gbc/CqrlogImproved
```

3. SRC component and version list

The following is the recorded SRC chain collected/built for this installation.

Component	Version	SRC / note
CqrlogImproved	CI (144) / b4b766c	OH1KH/CqrlogImproved
Qt qtbase	6.12.0	~/qt-src/qt-6.12/qtbase
Qt6Pas	6.2.10	Lazarus 4.4 qt6/cbindings
FPC	3.2.2	bootstrap/build tool
Lazarus/LCL	4.4	bootstrap + LCL SRC
Hamlib	5.0.0~git	hamlib-master
OpenSSL	3.5.5	openssl-3.5.5
MariaDB Connector/C	3.4.9	mariadb-connector-c-3.4.9
MariaDB Server	11.8.6	mariadb-server-11.8.6
libaio	0.3.113	libaio-0.3.113
libxcrypt	4.5.1	libxcrypt-4.5.1
numactl	2.0.19	numactl-2.0.19
liburing	2.14	liburing-2.14
fmt	12.1.0	fmt-12.1.0
libusb	1.0.29	libusb-1.0.29
zlib	1.3.1	zlib-1.3.1
libpng	1.6.57	libpng-1.6.57
expat	2.7.4	expat-2.7.4
pcre2	10.46	pcre2-10.46
ICU	78.2	icu-78.2
GLib	2.88.0	glib-2.88.0
D-Bus	1.16.2	dbus-1.16.2
systemd	259	systemd-259
Brotli	1.2.0	brotli-1.2.0
Graphite2	1.3.14	graphite2-1.3.14
HarfBuzz	12.3.2	harfbuzz-12.3.2
Fontconfig	2.17.1	fontconfig-2.17.1
libXau	1.0.11	libXau-1.0.11
libxkbcommon	1.13.1	xkbcommon-1.13.1
libXext	1.3.7	libXext-1.3.7
libglvnd	1.7.0	libglvnd-1.7.0
libXrender	0.9.12	libXrender-0.9.12
libXfixes	6.0.2	libXfixes-6.0.2
libXrandr	1.5.5	libXrandr-1.5.5

libXi	1.8.3	libXi-1.8.3
libXtst	SRC	X11 SRC chain
libXshmfcence	SRC	X11 SRC chain
GCC	15.2.0	gcc-15.2.0
glibc	2.43	glibc-2.43
GMP/MPFR/MPC/ISL	6.2.1 / 4.1.0 / 1.2.1 / 0.24	GCC prerequisite SRC
bash	5.3 + patches	CLI SRC
tar	1.35	CLI SRC
gawk	5.3.2	CLI SRC
gzip	1.14	CLI SRC
wget	1.25.0	CLI SRC
findutils	4.10.0	CLI SRC
coreutils	9.7	CLI SRC
dash	0.5.12	CLI SRC
procps-ng	4.0.4	CLI SRC
psmisc	23.7	CLI SRC
zip	3.0	CLI SRC
unzip	6.00	CLI SRC

4. Before building: safe checks

```
echo "===== COMPILERS ====="
gcc --version | head -1
g++ --version | head -1
cmake --version | head -1
make --version | head -1
git --version
pkg-config --version

echo
echo "===== FPC / LAZARUS ====="
fpc -iV
lazbuild --version
```

Expected result / what should appear: Real version numbers must appear, not "command not found". In the recorded build FPC was 3.2.2 and Lazarus/lazbuild was 4.4.

Note: This is a read-only check. It does not install anything.

```
echo "===== QT 6.12 SOURCE / QMAKE ====="
test -d "$HOME/qt-src/qt-6.12/qtbase" && echo "QT SRC OK"
test -x "$HOME/Qt/6.12/gcc_64/bin/qmake6" && \
"$HOME/Qt/6.12/gcc_64/bin/qmake6" --version || true
```

Expected result / what should appear: "QT SRC OK" should appear. If an already prepared Qt build tool is present, qmake should report Qt 6.12.x.

5. Build environment and paths

```
export BASE="$HOME/Programmas/cqrlog-src"
export SYSROOT="$BASE/sysroot"
export TOOLCHAIN="$BASE/toolchain"
export RUNTIME="$BASE/runtime"
export BUILD="$BASE/build"

export PATH="$TOOLCHAIN/bin:$RUNTIME/bin:$PATH"
export PKG_CONFIG_PATH="$RUNTIME/lib/pkgconfig:$RUNTIME/share/pkgconfig"
export LIBRARY_PATH="$RUNTIME/lib"
export LD_LIBRARY_PATH="$RUNTIME/lib"
export CMAKE_PREFIX_PATH="$RUNTIME"
export QT_PLUGIN_PATH="$RUNTIME/plugins"

printf '%s\n' "$PATH" "$PKG_CONFIG_PATH" "$LD_LIBRARY_PATH"
```

Expected result / what should appear: PATH should begin with the private TOOLCHAIN/RUNTIME paths. Library paths should point to ~/Programmas/cqrlog-src/runtime.

6. Checking the SRC collection with Python

```
python3 - <<'PY'
from pathlib import Path

roots = [
    Path.home() / "CqrlogImproved-SRC",
    Path.home() / "qt-src/qt-6.12/qtbase",
    Path.home() / "CqrlogImproved",
]

for p in roots:
    print(("OK   " if p.exists() else "MISS "), p)

src = Path.home() / "CqrlogImproved-SRC"
wanted = [
    "openssl-3.5.5",
    "mariadb-connector-c-3.4.9",
    "mariadb-server-11.8.6",
    "libaio-0.3.113",
    "libxcrypt-4.5.1",
    "numactl-2.0.19",
    "liburing-2.14",
    "fmt-12.1.0",
    "hamlib-master",
]

for name in wanted:
    p = src / name
    print(("OK   " if p.exists() else "MISS "), p)

PY
```

Expected result / what should appear: Every required directory should be prefixed with "OK". If "MISS" appears, that SRC tree is not present and the build should not be started.

7. Build order and dependency chain

The build was performed sequentially as one complete dependency chain. When a build failed, it was not continued from a questionable partial state: the problem was corrected and the relevant full build was run again.

1. Toolchain and base C/C++ environment: GCC 15.2.0 + GMP/MPFR/MPC/ISL; glibc 2.43.
2. Compression, XML, regex, Unicode and base system libraries.
3. Font/text stack: Brotli, Graphite2, HarfBuzz, Fontconfig.
4. X11/XCB stack, xkbcommon and OpenGL/GLVND.
5. Qt 6.12.0 qtbase with XCB.
6. Lazarus Qt6 cbindings -> libQt6Pas.
7. OpenSSL -> MariaDB Connector/C -> libaio/libxcrypt/numactl/liburing/fmt -> MariaDB Server.
8. Hamlib.
9. CqrlogImproved Qt6.
10. Private runtime install, data migration and .desktop integration.

7.1. Generic SRC build patterns

```
# Autotools-style project
cd /path/to/SRC
rm -rf build
mkdir build && cd build
../configure --prefix="$RUNTIME"
make -j"$(nproc)"
make install
```

Expected result / what should appear: Libraries/binaries should be created under \$RUNTIME/lib and \$RUNTIME/bin, with headers under \$RUNTIME/include.

Note: Some projects need additional flags; use that project's SRC README and configure --help.

```
# CMake-style project
cmake -S /path/to/SRC -B "$BUILD/package" \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$RUNTIME"
cmake --build "$BUILD/package" --parallel
cmake --install "$BUILD/package"
```

Expected result / what should appear: The build must finish without ERROR; after installation, files should appear below \$RUNTIME.

```
# Meson/Ninja-style project
meson setup "$BUILD/package" /path/to/SRC \
  --prefix="$RUNTIME" --buildtype=release
ninja -C "$BUILD/package"
ninja -C "$BUILD/package" install
```

Expected result / what should appear: Meson should finish with a configuration summary, and Ninja should finish without a failed target.

8. Building Qt 6.12.0 from SRC

```
export BASE="$HOME/Programmas/cqrlog-src"
export RUNTIME="$BASE/runtime"
export QTSRC="$HOME/qt-src/qt-6.12/qtbase"
export QTBUILD="$BASE/build/qtbase-6.12"

rm -rf "$QTBUILD"
mkdir -p "$QTBUILD"
cd "$QTBUILD"

"$QTSRC/configure" \
  -prefix "$RUNTIME" \
  -release \
  -opensource -confirm-license \
  -nomake examples -nomake tests \
  -xcb
```

Expected result / what should appear: The configure step must finish without a fatal error. XCB/platform plugin support should be enabled.

Note: In this build, the Qt X11/XCB dependencies were built from SRC before Qt.

```
cmake --build "$QTBUILD" --parallel
cmake --install "$QTBUILD"

"$RUNTIME/bin/qmake6" --version 2>/dev/null || \
"$RUNTIME/bin/qmake" --version
```

Expected result / what should appear: Qt 6.12.0 should be reported, with qmake coming from the private runtime. This is the critical checkpoint before Qt6Pas.

9. Building Qt6Pas against the private Qt6

The first CqrlogImproved Qt6 build stopped with "cannot find -lQt6Pas". The fix was to build the Lazarus Qt6 cbindings directly against the same private Qt 6.12 runtime.

```
export RUNTIME="$HOME/Programmas/cqrlog-src/runtime"
export QTPAS_SRC="/usr/lib/lazarus/4.4/lcl/interfaces/qt6/cbindings"
export QTPAS_BUILD="$HOME/Qt6Pas-build"

rm -rf "$QTPAS_BUILD"
cp -a "$QTPAS_SRC" "$QTPAS_BUILD"
cd "$QTPAS_BUILD"

export LD_LIBRARY_PATH="$RUNTIME/lib"
"$RUNTIME/bin/qmake6" Qt6Pas.pro 2>/dev/null || \
"$RUNTIME/bin/qmake" Qt6Pas.pro
make -j"$(nproc)"
```

Expected result / what should appear: A `libQt6Pas.so` library should be created. The recorded final build produced `libQt6Pas.so.6.2.10`.

Note: If `qmake` is not named `qmake6`, use `$RUNTIME/bin/qmake`.

```
mkdir -p "$RUNTIME/lib"
cp -a libQt6Pas.so* "$RUNTIME/lib/"

ls -l "$RUNTIME/lib"/libQt6Pas.so*
```

Expected result / what should appear: The real library and its symlink chain should be visible, including `libQt6Pas.so` and the soname link for 6.2.10.

10. OpenSSL and MariaDB chain

10.1. OpenSSL 3.5.5

```
export RUNTIME="$HOME/Programmas/cqrlog-src/runtime"
cd "$HOME/CqrlogImproved-SRC/openssl-3.5.5"

make clean 2>/dev/null || true
./Configure linux-x86_64 \
  --prefix="$RUNTIME" \
  --openssldir="$RUNTIME/ssl" \
  shared
make -j"$(nproc)"
make install_sw

ls -l "$RUNTIME/lib"/libssl.so* "$RUNTIME/lib"/libcrypto.so*
```

Expected result / what should appear: `libssl.so.3` and `libcrypto.so.3` should be present in the private runtime.

Note: The `CqrlogImproved Synapse/OpenSSL` path dynamically looks for `libssl.so` and `libcrypto.so`.

10.2. MariaDB Connector/C 3.4.9

```
export BASE="$HOME/Programmas/cqrlog-src"
export RUNTIME="$BASE/runtime"
SRC="$HOME/CqrlogImproved-SRC/mariadb-connector-c-3.4.9"
BLD="$BASE/build/mariadb-connector-c"

rm -rf "$BLD"
cmake -S "$SRC" -B "$BLD" \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$RUNTIME" \
  -DCMAKE_PREFIX_PATH="$RUNTIME"
cmake --build "$BLD" --parallel
cmake --install "$BLD"
```

Expected result / what should appear: MariaDB client libraries and headers should be installed under `$RUNTIME`.

10.3. MariaDB server helper dependencies

libaio 0.3.113

```
cd "$HOME/CqrlogImproved-SRC/libaio-0.3.113"
make clean 2>/dev/null || true
make prefix="$RUNTIME" install
```

Expected result / what should appear: A libaio library should appear under \$RUNTIME/lib.

libxcrypt 4.5.1

```
cd "$HOME/CqrlogImproved-SRC/libxcrypt-4.5.1"
make distclean 2>/dev/null || true
./configure --prefix="$RUNTIME" --disable-werror
make -j"$(nproc)"
make install
```

Expected result / what should appear: The build should complete without being stopped by warnings treated as errors.

numactl 2.0.19

```
cd "$HOME/CqrlogImproved-SRC/numactl-2.0.19"
make distclean 2>/dev/null || true
./configure --prefix="$RUNTIME"
make -j"$(nproc)"
make install
```

Expected result / what should appear: A libnuma library should appear under \$RUNTIME/lib.

liburing 2.14

```
cd "$HOME/CqrlogImproved-SRC/liburing-2.14"
make clean 2>/dev/null || true
./configure --prefix="$RUNTIME"
make -j"$(nproc)"
make install
```

Expected result / what should appear: A liburing library should appear under \$RUNTIME/lib.

10.4. fmt 12.1.0

```
export BASE="$HOME/Programmas/cqrlog-src"
export RUNTIME="$BASE/runtime"
SRC="$HOME/CqrlogImproved-SRC/fmt-12.1.0"
BLD="$BASE/build/fmt"

rm -rf "$BLD"
cmake -S "$SRC" -B "$BLD" \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$RUNTIME" \
  -DFMT_TEST=OFF
cmake --build "$BLD" --parallel
cmake --install "$BLD"
```

Expected result / what should appear: fmt libraries/headers should appear under \$RUNTIME. Tests are not built.

10.5. MariaDB Server 11.8.6

```
export BASE="$HOME/Programmas/cqrlog-src"
export RUNTIME="$BASE/runtime"
SRC="$HOME/CqrlogImproved-SRC/mariadb-server-11.8.6"
BLD="$BASE/build/mariadb-server"

rm -rf "$BLD"
cmake -S "$SRC" -B "$BLD" \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$RUNTIME" \
  -DCMAKE_PREFIX_PATH="$RUNTIME" \
  -DWITH_UNIT_TESTS=OFF
cmake --build "$BLD" --parallel
cmake --install "$BLD"

ls -l "$RUNTIME/bin"/mariadb* "$RUNTIME/bin"/mariabdd 2>/dev/null
```

Expected result / what should appear: At least mariadb, mariadb-dump and mariabdd should be present in the private runtime. In the recorded build this was stage 68/71.

```
ln -sf mariabdd "$RUNTIME/bin/mysqld"
ln -sf mariadb "$RUNTIME/bin/mysql"
ln -sf mariadb-dump "$RUNTIME/bin/mysqldump"

ls -l "$RUNTIME/bin"/mysql "$RUNTIME/bin"/mysqld "$RUNTIME/bin"/mysqldump
```

Expected result / what should appear: Three symlinks should be visible and should point to the source-built MariaDB program names.

11. Building Hamlib from SRC

```
export RUNTIME="$HOME/Programmas/cqrlog-src/runtime"
cd "$HOME/CqrlogImproved-SRC/hamlib-master"

./bootstrap 2>/dev/null || true
./configure --prefix="$RUNTIME"
make -j"$(nproc)"
make install

"$RUNTIME/bin/rigctld" --version | head -2
ls -l "$RUNTIME/lib"/libhamlib.so*
```

Expected result / what should appear: Hamlib 5.0.0~git should be reported, with rigctld and libhamlib.so in the runtime. In the recorded build this was stage 69/71.

12. Building CqrlogImproved Qt6

The upstream CqrlogImproved Makefile uses lazbuild with --ws=qt6 for the Qt6 target. Therefore libQt6Pas and the Qt6 plugin environment must be ready before this step.

```
export RUNTIME="$HOME/Programmas/cqrlog-src/runtime"
export CQR="$HOME/CqrlogImproved"

export PATH="$RUNTIME/bin:$PATH"
export LD_LIBRARY_PATH="$RUNTIME/lib"
export LIBRARY_PATH="$RUNTIME/lib"
export QT_PLUGIN_PATH="$RUNTIME/plugins"
export QT_QPA_PLATFORM=xcb

cd "$CQR"
make clean
make cqrlog_qt6
```

Expected result / what should appear: A successful final sequence includes: Linking .../CqrlogImproved/src/cqrlog, warnings but no fatal error, strip src/cqrlog, gzip tools/cqrlog.1, and the src/cqrlog executable exists.

Note: In the recorded build this was 70/71 and produced an approximately 6.3 MB cqrlog binary.

```
ls -lh "$HOME/CqrlogImproved/src/cqrlog"
file "$HOME/CqrlogImproved/src/cqrlog"
```

Expected result / what should appear: An executable x86-64 ELF binary should be reported, with a file size of several MB.

13. Installing into the private runtime

```
export BASE="$HOME/Programmas/cqrlog-src"
export RUNTIME="$BASE/runtime"
export CQR="$HOME/CqrlogImproved"

export PATH="$RUNTIME/bin:$PATH"
export LD_LIBRARY_PATH="$RUNTIME/lib"
export LIBRARY_PATH="$RUNTIME/lib"
export QT_PLUGIN_PATH="$RUNTIME/plugins"
export QT_QPA_PLATFORM=xcb

cd "$CQR"
make install DESTDIR="$RUNTIME"

ln -sf mariadb "$RUNTIME/bin/mysqld"
ln -sf mariadb "$RUNTIME/bin/mysql"
ln -sf mariadb-dump "$RUNTIME/bin/mysqldump"
```

Expected result / what should appear: The install should create \$RUNTIME/bin/cqrlog, \$RUNTIME/share/cqrlog/..., \$RUNTIME/share/icons/cqrlog.png, plus the MariaDB symlinks.

Note: The upstream Makefile treats DESTDIR as the installation prefix: bindir=\$DESTDIR/bin and datadir=\$DESTDIR/share/cqrlog.

```
echo "==== RUNTIME AUDIT ====="
ldd "$RUNTIME/bin/cqrlog" | grep 'not found' || true
ls -lh "$RUNTIME/bin/cqrlog"
ls -ld "$RUNTIME/share/cqrlog" "$RUNTIME/plugins" 2>/dev/null
```

Expected result / what should appear: No "not found" lines should appear. cqrlog and share/cqrlog should exist.

14. First launch

```
export RUNTIME="$HOME/Programmas/cqrlog-src/runtime"
export PATH="$RUNTIME/bin:$PATH"
export LD_LIBRARY_PATH="$RUNTIME/lib"
export QT_PLUGIN_PATH="$RUNTIME/plugins"
export QT_QPA_PLATFORM=xcb

cd "$RUNTIME/bin"
./cqrlog
```

Expected result / what should appear: The white CqrlogImproved Qt6 window should open. In this manual the white Qt6 appearance is the correct native/base design.

15. Migrating old CQRLOG data and ALL configuration

In this migration the complete old environment was under ~/.config/cqrlog2, while the new program uses ~/.config/cqrlog. The old environment contained cqrlog001, cqrlog002, cqrlog003 and cqrlog_common, plus 1local.cfg, 2local.cfg, 3local.cfg, local.cfg and cqrlog_login.cfg.

15.1. Check that the CQRLOG database process is not active

```
echo "===== BEFORE BACKUP ====="

pgrep -af 'cqrlog|mariadb|mysqld' || \
echo "NO CQRLOG OR MARIADB PROCESSES"
```

Expected result / what should appear: If only an unrelated system MariaDB is visible, do not touch it. The CQRLOG-local process must not be holding ~/.config/cqrlog/database open.

15.2. Backup the current new environment

```
echo "===== FULL CQRLOG BACKUP ====="

BACKUP="$HOME/CQRLOG-BACKUP-2026-08-21-1800"
mkdir -p "$BACKUP" || exit 1

cp -a "$HOME/.config/cqrlog" \
  "$BACKUP/cqrlog-config-and-database" || exit 1

cp -a "$HOME/.local/share/applications/cqrlog.desktop" \
  "$BACKUP/" 2>/dev/null || true

cp -a "$HOME/.local/share/applications/cqrlog2.desktop" \
  "$BACKUP/" 2>/dev/null || true

du -sh "$BACKUP" "$BACKUP/cqrlog-config-and-database"
```

Expected result / what should appear: In the recorded migration both size lines were about 149M, confirming a full copy of ~/.config/cqrlog.

15.3. Full backup of the old cqrlog2 environment

```
echo "===== FULL OLD CQRLOG2 BACKUP ====="

BACKUP="$HOME/CQRLOG-BACKUP-2026-08-21-1800"
mkdir -p "$BACKUP"

cp -a "$HOME/.config/cqrlog2" \
  "$BACKUP/cqrlog2-full"

du -sh \
  "$BACKUP/cqrlog-config-and-database" \
  "$BACKUP/cqrlog2-full"

find "$BACKUP/cqrlog2-full/database" \
  -maxdepth 1 -mindepth 1 -type d \
  -printf '%f\n' | sort
```

Expected result / what should appear: In the recorded migration cqrlog2-full was about 154M and listed cqrlog001, cqrlog002, cqrlog003 and cqrlog_common.

15.4. Full migration cqrlog2 -> cqrlog

```
echo "===== FULL CQRLOG2 -> CQRLOG MIGRATION ====="

SRC="$HOME/.config/cqrlog2"
DST="$HOME/.config/cqrlog"
STAMP="$(date +%Y%m%d-%H%M%S)"
OLD="$HOME/.config/cqrlog-before-migration-$STAMP"

RUNNING="$(pgrep -u "$USER" -af 'cqrlog|mariadb|mysqld' || true)"

if [ -n "$RUNNING" ]; then
    echo "MIGRATION NOT STARTED - PROCESSES STILL RUNNING:"
    printf '%s\n' "$RUNNING"
else
    if [ -d "$DST" ]; then
        mv "$DST" "$OLD"
        echo "Previous cqrlog saved as: $OLD"
    fi

    cp -a "$SRC" "$DST"

    du -sh "$DST"
    find "$DST/database" -maxdepth 1 -mindepth 1 \
        -type d -printf '%f\n' | sort
    find "$DST" -maxdepth 1 -type f -name '*local.cfg' \
        -printf '%f\n' | sort
fi
```

Expected result / what should appear: The successful result was approximately 154M ~/.config/cqrlog and the entries cqrlog001, cqrlog002, cqrlog003, cqrlog_common, 1local.cfg, 2local.cfg, 3local.cfg and local.cfg.

15.5. Identity check

```
diff -qr "$HOME/.config/cqrlog2" "$HOME/.config/cqrlog" || true
```

Expected result / what should appear: If nothing is printed, both directories are identical at the file/structure level. That was the successful recorded result.

16. Integrating into the Amateur Radio Programs menu

```
echo "===== NEW CQRLOG MENU ENTRY ====="

RUNTIME="$HOME/Programmas/cqrlog-src/runtime"
DESKTOP="$HOME/.local/share/applications/cqrlog.desktop"

[ -f "$HOME/.local/share/applications/cqrlog2.desktop" ] && \
mv "$HOME/.local/share/applications/cqrlog2.desktop" \
"$HOME/.local/share/applications/cqrlog2.desktop.old"

ICON="$(find "$RUNTIME/share" -type f -iname 'cqrlog*.png' 2>/dev/null | head -1)"
[ -z "$ICON" ] && ICON="$HOME/CqrlogImproved/images/cqrlog.png"

cat > "$DESKTOP" <<EOF
[Desktop Entry]
Type=Application
Name=CqrlogImproved
Comment=Amateur Radio Logging Program
Exec=/usr/bin/env LD_LIBRARY_PATH=$RUNTIME/lib QT_PLUGIN_PATH=$RUNTIME/plugins QT_QPA_PLATFORM=xcb
PATH=$RUNTIME/bin:/usr/local/bin:/usr/bin:/bin $RUNTIME/bin/cqrlog
Path=$RUNTIME/bin
Icon=$ICON
Terminal=false
Categories=Network;HamRadio;
Keywords=Hamradio;Radio;Logger;Logbook;
StartupNotify=true
EOF

chmod +x "$DESKTOP"
grep -E '^(Name|Exec|Path|Icon|Categories)= ' "$DESKTOP"
```

Expected result / what should appear: The output should show Name=CqrlogImproved, Exec pointing to .../runtime/bin/cqrlog, Path=.../runtime/bin, Icon=.../runtime/share/icons/cqrlog.png, and Categories=Network;HamRadio;

17. Final verification

Open CqrlogImproved from the application menu and leave it running.

```
echo "===== CQRLOG FROM MENU ====="

PID="$(pgrep -n -x cqrlog)"
echo "PID=$PID"

if [ -n "$PID" ]; then
    echo -n "BINARY: "
    readlink -f "/proc/$PID/exe"

    echo -n "WORKING DIRECTORY: "
    readlink -f "/proc/$PID/cwd"
fi
```

Expected result / what should appear: The successful recorded result was BINARY: /home/yl3gbc/Programmas/cqrlog-src/runtime/bin/cqrlog and WORKING DIRECTORY: /home/yl3gbc/Programmas/cqrlog-src/runtime/bin.

```
echo "===== CQRLOG DATABASES ====="
find "$HOME/.config/cqrlog/database" \
-maxdepth 1 -mindepth 1 -type d \
-printf '%f\n' | sort
```

Expected result / what should appear: All four should appear: cqrlog001, cqrlog002, cqrlog003 and cqrlog_common.

17.1. Process check

```
pgrep -af 'cqrlog|mariadb|mysqld'
```

Expected result / what should appear: The running CqrlogImproved process should be visible. The CQRLOG-local database process uses ~/.config/cqrlog/database and its own socket/port. If /usr/sbin/mariadb is visible, do not kill it automatically; first verify whether it actually belongs to CQRLOG.

Note: In the recorded run, the CQRLOG-local database used port 64000.

18. Troubleshooting

18.1. "cannot find -lQt6Pas"

Meaning: the Pascal Qt6 widgetset link cannot find libQt6Pas. The correct fix is to rebuild the Lazarus qt6/cbindings against the same private Qt6, not to insert an unrelated prebuilt binary library.

```
ls -l "$RUNTIME/lib"/libQt6Pas.so*
echo "$LD_LIBRARY_PATH"
"$RUNTIME/bin/qmake6" --version 2>/dev/null || "$RUNTIME/bin/qmake" --version
```

Expected result / what should appear: The libQt6Pas.so links, the \$RUNTIME/lib environment path and Qt 6.12.0 should be visible.

18.2. Qt platform plugin / xcb problem

```
export QT_PLUGIN_PATH="$RUNTIME/plugins"
export QT_QPA_PLATFORM=xcb

find "$RUNTIME/plugins" -maxdepth 2 -type f \
-name 'libqxcb.so' -print
```

Expected result / what should appear: platforms/libqxcb.so should be found. If it is missing, the Qt/XCB build stage is incomplete.

18.3. Cqrlog is already running

```
pgrep -af 'cqrlog|cqrlog2'
```

Expected result / what should appear: If an existing cqrlog process is shown, a second copy may refuse to start with "Cqrlog is already running !!". Close the already running application first.

18.4. MariaDB process safety

```
ps -p 1537 -o user=,pid=,ppid=,lstart=,args=
tr '\0' ' ' < /proc/1537/cmdline 2>/dev/null; echo
```

Expected result / what should appear: If the process belongs to mysql and is /usr/sbin/mariadb without CQRLOG datadir arguments, it may be an independent system server. Do not stop it merely because its process name is mariadb.

19. Update procedure

Before building a newer CqrlogImproved commit, keep a data backup. On 2026-08-21 the newest recorded main-branch commit was CI_(144)_, b4b766c186d95dbaad3da732527c9cfa6f581976.

```
cd "$HOME/CqrlogImproved"
git status --short
git fetch origin
git log --oneline -5 origin/main
```

Expected result / what should appear: The local working-tree status and the newest origin/main commits should be shown. Do not accidentally lose local changes before rebuilding.

```
cd "$HOME/CqrlogImproved"
git pull --ff-only

export RUNTIME="$HOME/Programmas/cqrlog-src/runtime"
export PATH="$RUNTIME/bin:$PATH"
export LD_LIBRARY_PATH="$RUNTIME/lib"
export QT_PLUGIN_PATH="$RUNTIME/plugins"
export QT_QPA_PLATFORM=xcb

make clean
make cqrlog_qt6
make install DESTDIR="$RUNTIME"
```

Expected result / what should appear: The process should end with a new `src/cqrlog` and an updated `$RUNTIME/bin/cqrlog`. The data directory `~/config/cqrlog` is not deleted.

20. SRC provenance audit

The objective is not only that the program starts, but also that the runtime provenance is understood. After the final build, check the program binary, library paths and source-built runtime artifacts.

```
python3 - <<'PY'
from pathlib import Path
import subprocess

runtime = Path.home() / "Programmas/cqrlog-src/runtime"
exe = runtime / "bin/cqrlog"

print("CQRLOG:", exe, "OK" if exe.exists() else "MISSING")
print("Qt6Pas:", list((runtime/"lib").glob("libQt6Pas.so*"))[:4])
print("Hamlib:", list((runtime/"lib").glob("libhamlib.so*"))[:4])
print("MariaDB:", [p.name for p in (runtime/"bin").glob("maria*")][:8])

if exe.exists():
    p = subprocess.run(["ldd", str(exe)], text=True, capture_output=True)
    missing = [x for x in p.stdout.splitlines() if "not found" in x]
    print("MISSING LIBS:", len(missing))
    for x in missing:
        print(x)
PY
```

Expected result / what should appear: `CQRLOG: ... OK` should appear, followed by `Qt6Pas/Hamlib/MariaDB` artifacts and `MISSING LIBS: 0`.

Appendix A. Final 71/71 stages

These are the precisely recorded final stages of the successful build. Earlier stages covered the toolchain, base libraries, X11/XCB and Qt prerequisites.

Stage	Component
50/71	libxkbcommon 1.13.1
51/71	libXext 1.3.7
52/71	libglvnd 1.7.0
53/71	libXrender 0.9.12
54/71	libXfixes 6.0.2
55/71	libXrandr 1.5.5
56/71	libXi 1.8.3
57/71	libXtst
58/71	libXshmfence
59/71	Qt 6.12.0 qtbase
60/71	Qt6Pas 6.2.10
61/71	OpenSSL 3.5.5
62/71	MariaDB Connector/C 3.4.9
63/71	libaio 0.3.113
64/71	libxcrypt 4.5.1 (--disable-werror)

65/71	numactl 2.0.19
66/71	liburing 2.14
67/71	fmt 12.1.0
68/71	MariaDB Server 11.8.6
69/71	Hamlib 5.0.0~git
70/71	CqrlogImproved Qt6 compilation
71/71	Private runtime install, audit and launch

Appendix B. Restoring backups

If you need to roll back after migration, first close CqrlogImproved and its local database process. Then restore the previously saved configuration copy.

```
pgrep -u "$USER" -af 'cqrlog|mariadb|mysqld' || true
```

Expected result / what should appear: Verify that the CQRLOG-local database is no longer keeping data files open.

```
STAMP="$(date +%Y%m%d-%H%M%S)"
mv "$HOME/.config/cqrlog" \
  "$HOME/.config/cqrlog-failed-$STAMP"

cp -a \
  "$HOME/CQRLOG-BACKUP-2026-08-21-1800/cqrlog2-full" \
  "$HOME/.config/cqrlog"

diff -qr \
  "$HOME/CQRLOG-BACKUP-2026-08-21-1800/cqrlog2-full" \
  "$HOME/.config/cqrlog" || true
```

Expected result / what should appear: If diff prints nothing, the restored environment is identical to the saved backup.

Conclusion

The successful final state is CqrlogImproved Qt6 launching from the user application menu, using the complete migrated CQRLOG environment with all three logs and all configuration, while preserving separate backups. The build principle is source-first: final runtime components are built from SRC trees and installed into an isolated runtime.

References: OH1KH/CqrlogImproved README and Makefile; Lazarus Qt6 cbindings source. The upstream CqrlogImproved Makefile provides the make cqrlog_qt6 target, while make install DESTDIR=... installs the binary and share resources into the selected private prefix.